



操作系统课程设计





课程安排与要求

2026.7.6 9:00-11:30	19周周一 13:30-16:00	分组，各小组选题，明确小组分工。	信息楼318 (计23) 信息楼211 (软23)
2026.7.7 9:00-11:30	19周周二 13:30-16:00	确定技术路线，拟定初步方案。	信息楼318 (计23) 信息楼211 (软23)
2026.7.8 9:00-11:30	19周周三 13:30-16:00	系统设计。	信息楼318 (计23) 信息楼211 (软23)
2026.7.9 9:00-11:30	19周周四 13:30-16:00	系统实现。	信息楼318 (计23) 信息楼211 (软23)
2026.7.10 9:00-11:30	19周周五 13:30-16:00	系统实现。	信息楼318 (计23) 信息楼211 (软23)
2026.7.13 9:00-11:30	20周周一 13:30-16:00	系统测试、系统完善与改进。	信息楼318 (计23) 信息楼211 (软23)
2026.7.14 9:00-11:30	20周周二 13:30-16:00	撰写课程设计报告。	信息楼318 (计23) 信息楼211 (软23)
2026.7.15 9:00-11:30	20周周三 13:30-16:00	演示系统，汇报成果，打印并提交课程报告。	信息楼318 (计23) 信息楼211 (软23)

- 不超过3名同学分为一组，每组同学在实验题目中**任选一个题目**实现。
- 19周 周二 24:00 之前完成学习通平台活动中选择分组。



- 操作系统：Linux操作系统，版本可以选择Ubuntu、RHEL 红帽系、银河麒麟 KylinOS、欧拉等系统。
- Linux内核理解：各种版本的Linux 内核都可以，鼓励同学下载Linux Kernel 6.18（ <https://kernel.org/> ）或者线上阅读代码：
kernel/git/stable/linux.git - Linux kernel stable tree
- 编程环境：使用VC、C、Java、C++等程序设计语言，借助这些语言环境来完成实验。

- kernel 操作系统核心服务，进程、信号、系统调用、定时器
 - kernel/sched: 进程调度 (fair.c/core.c/rt.c) , kernel/fork.c: 进程创建 fork/exec, kernel/signal.c: 信号处理, kernel/timer.c: 内核定时器, kernel/syscall.c: 系统调用入口
- mm 内存管理子系统，虚拟内存、物理内存分配、页表、缓存、swap
- fs 文件系统子系统
 - fs/ext2: ext2 文件系统 fs/btrfs/xfs: 新型文件系统 fs/proc: proc 虚拟文件系统 (ps、top 读取进程信息) fs/sysfs: 硬件 sys 文件系统
- drivers 硬件驱动统一存放
 - drivers/net: 网卡驱动, drivers/gpu: 显卡, drivers/tty: 串口、终端控制台, drivers/block: 磁盘块设备, drivers/usb: USB 驱动
- Makefile: 内核总编译脚本; Kconfig: 全局配置菜单; LICENSE: 开源协议 GPL2; MAINTAINERS: 各模块维护者清单

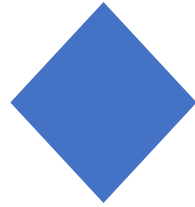
- 成绩=30%平时成绩+70%实践成绩（70%系统演示+30%课程报告）
- 20周周二或者周三（7月15日）进行系统演示（必须）
- 提交纸质版本的课程设计报告（必须）+成绩考核表（必须）
- 提交电子版报告+源代码（学习通平台）

课程报告和代码

计231-3

作答时间：07-06 09:00至 07-19 23:55

- 说明：系统演示或课程设计报告缺少一项均没有成绩

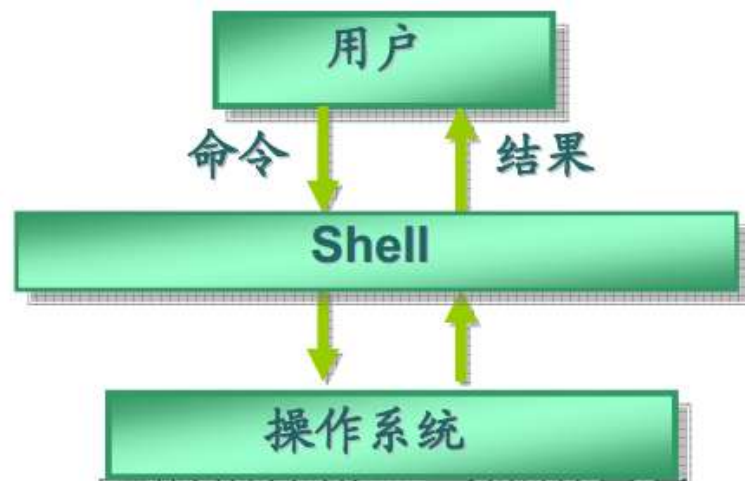


实验一

Shell 编程

- 学习如何编写一个Linux的外壳——Shell，即命令解释程序。学习怎样接受命令、解释命令、执行命令，特别是采用创建子进程的方式来执行一个程序，以及父进程如何继续子进程的工作。
- 任务要求：
 - 理解和掌握Linux中shell的原理和实现；
 - 编写一个C语言程序作为Linux内核的shell命令行解释程序；
 - 实现Shell的ls、cd、cat、grep、echo、type、history、alias、命令补全、后台运行、管道与重定向的功能。

- Shell 是一种具备特殊功能的程序，它是介于使用者和 UNIX/Linux 操作系统之核心程序（kernel）间的一个接口。
- 为了对用户屏蔽内核的复杂性，也为了保护内核以免用户误操作造成损害，在内核的周围建了一个外壳(shell)。用户向shell提出请求，shell解释并将请求传给内核。

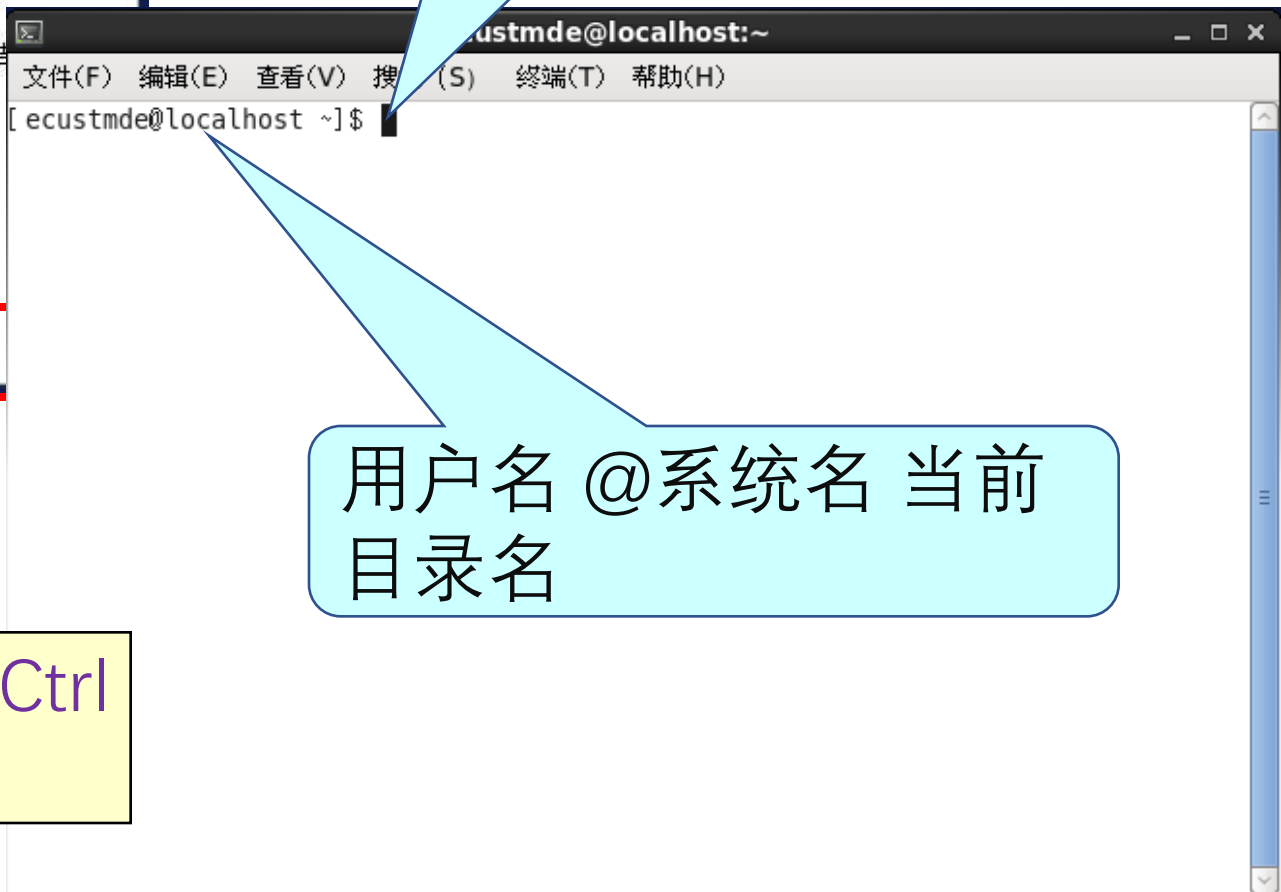




Shell提示符

\$: 普通用户

#: 超级用户



用户名 @系统名 当前
目录名

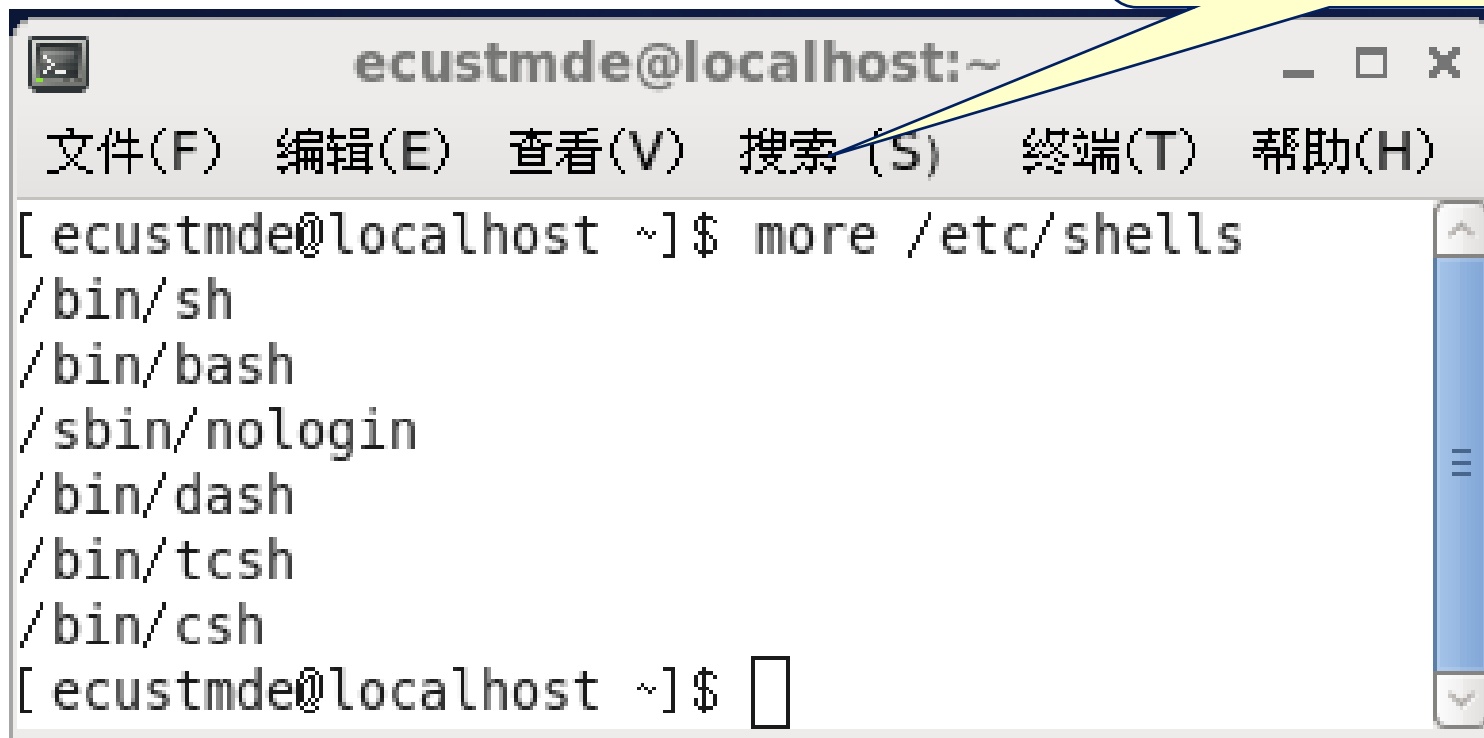
退出: \$exit或者Ctrl
+D

• Shell种类

- Bash(以及早期的sh): 来自于AT&T贝尔实验室Bourne创建的sh, 在GNU项目的资助下Brain Fox 开发了bash, 之后Chet Ramey使其得到发展。
- tcsh(以及早期的csh): 是C Shell(csh)的开放源代码版本, 采用的语法类似于 C 编程语言, 被大多数Berkeley UNIX系统作为默认Shell。
- Korn shell(ksh): AT&T贝尔实验室的David Korn开发, 它集合了C shell和Bourne shell的优点, 并且和Bourne shell完全兼容, 是UNIX System V系统默认的Shell。

- 在/etc/shells文件中，列出了目前系统可以使用的shell。并且给出了这些shell程序的位置。

在终端屏幕按屏
显示文本文件



```
ecustmde@localhost:~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[ecustmde@localhost ~]$ more /etc/shells  
/bin/sh  
/bin/bash  
/sbin/nologin  
/bin/dash  
/bin/tcsh  
/bin/csh  
[ecustmde@localhost ~]$
```

- 察看当前系统使用的Shell类型，输入命令：

```
[ecustmde@localhost ~]$ echo $SHELL  
/bin/bash
```

- 说明：echo命令的作用是在 显示器上显示一段文字
- 切换Shell类型

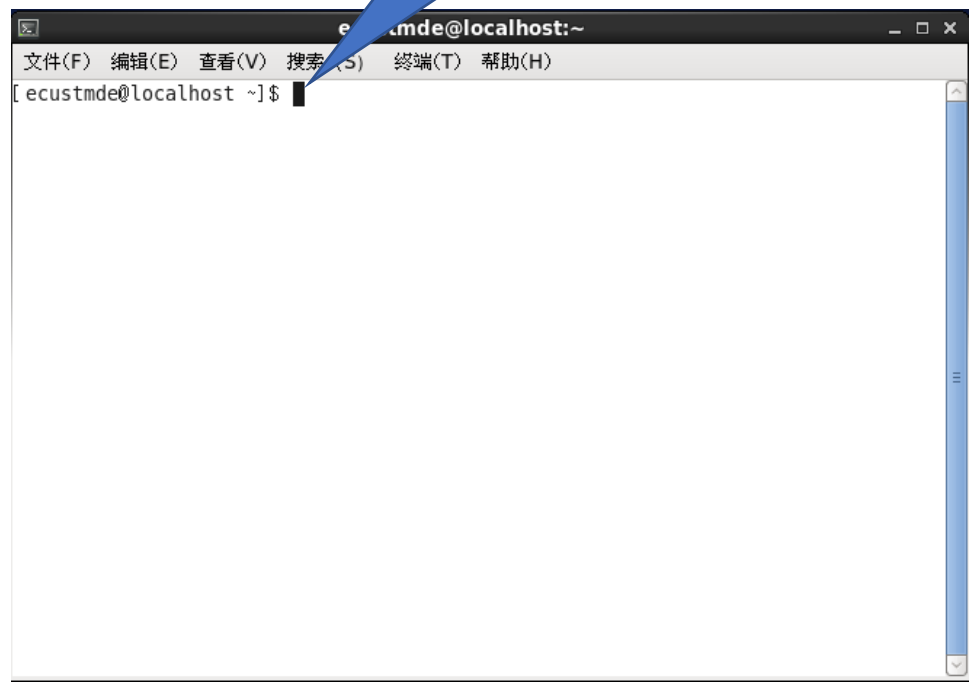
```
. ecustmde@localhost ~]$ tcsh  
. ecustmde@localhost ~]$ █
```

- 命令解释执行
- 命令补全
- 查阅历史功能
- 别名功能
- 后台处理
- I/O重定向
- 通信管道建立

(1)Shell命令解释及执行

- 用户登录成功后，实际进入Shell，它遵循一定的语法将输入的命令解释并传递给系统。
- 命令格式：
 - # 命令名 [选项] [参数]

输入命令



- 选项：更改命令的行为，形式为：-字母；
 - 例如：`#ls -l`
- 参数：命令使用的一条额外信息；
 - 例如：`#cat /etc/password`
- 环境变量：对用户Shell可能有用的信息；
 - 例如：`#echo $HOME`
- 元字符：对Shell有特殊意义的字符；
 - 例如：`#cat /etc/password |sort`

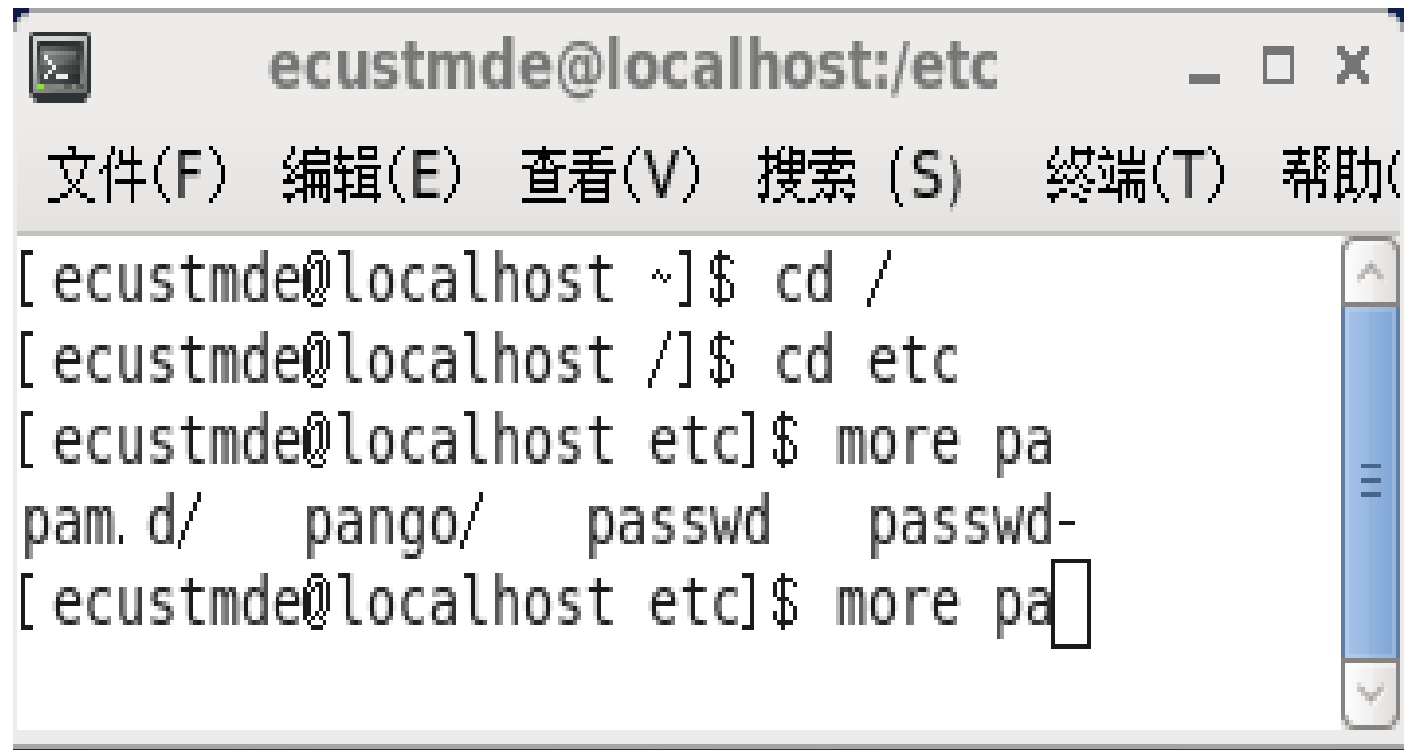
(2)命令补全功能

- 所谓“命令补全”是指在用户输入命令的时候，无需输入完整的命令行，Linux系统的shell会自动查找出最符合的命令名称，供用户选择。这样的功能可以帮用户节省输入长串命令的时间。
- 例如，在/root目录下有一个testexecvp.c文件，如果想查看其中的内容，并不需要完整输入该文件的名称，而只要输入开头的几个字母，即输入“more /root/teste”，然后按下Tab键一次，Linux的bash会自动补足完整的命令（more /root/testexecvp.c）。

- 例如:

- #echo Fi<tab> #echo File
- #cd ~ro<tab> #cd root/ 补全根用户的主目录
- #echo \$OS<tab> #echo \$OSTYPE

- 如果系统中有多文件都与输入的前缀相同，那么当用户连续按下2次Tab键时，系统会显示当前目录下所有具有相同前缀的文件名称，供用户选择。
- 例如，输入“more pa”后按2次Tab键，结果如图下所示。



A terminal window titled "ecustmde@localhost:/etc" with a menu bar containing "文件(F)", "编辑(E)", "查看(V)", "搜索 (S)", "终端(T)", and "帮助(H)". The terminal shows the following sequence of commands and output:

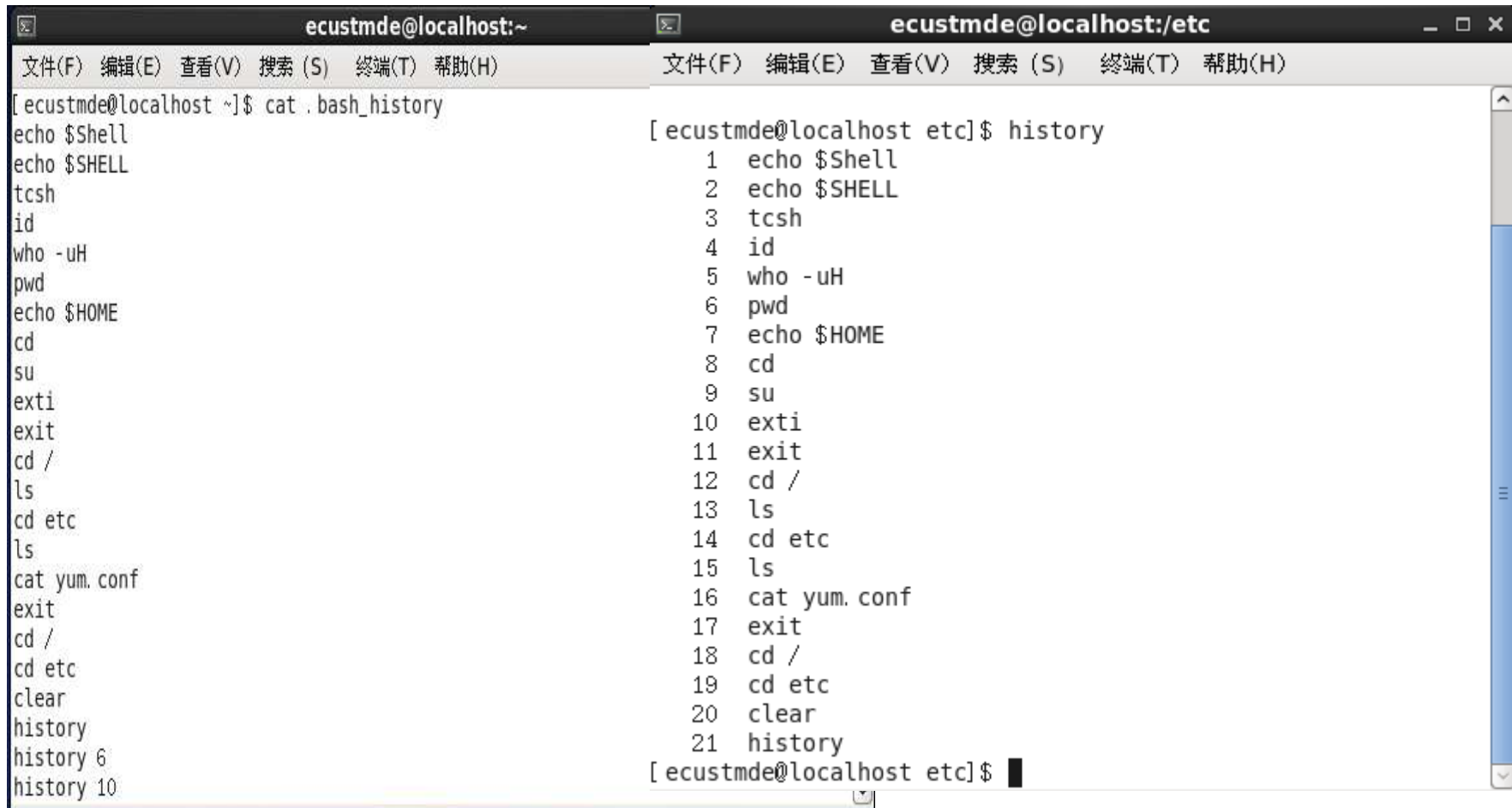
```
[ecustmde@localhost ~]$ cd /  
[ecustmde@localhost /]$ cd etc  
[ecustmde@localhost etc]$ more pa  
pam.d/    pango/    passwd    passwd-  
[ecustmde@localhost etc]$ more pa
```

The last line shows the command "more pa" with a cursor at the end, indicating that the terminal is waiting for further input or a second Tab press to trigger completion.

(3) 查阅历史记录

- history命令：每当用户输入的命令并按下Enter后，都会被记录在命令记录表中，默认情况下，bash默认使用的命令记录表文件为用户主目录下的.bash_history（文件名前面的“.”表示这是一个隐藏文件）文件。
- 可以使用环境变量HISTSIZE来定义命令记录表的条数，默认的记录条数为1000条。

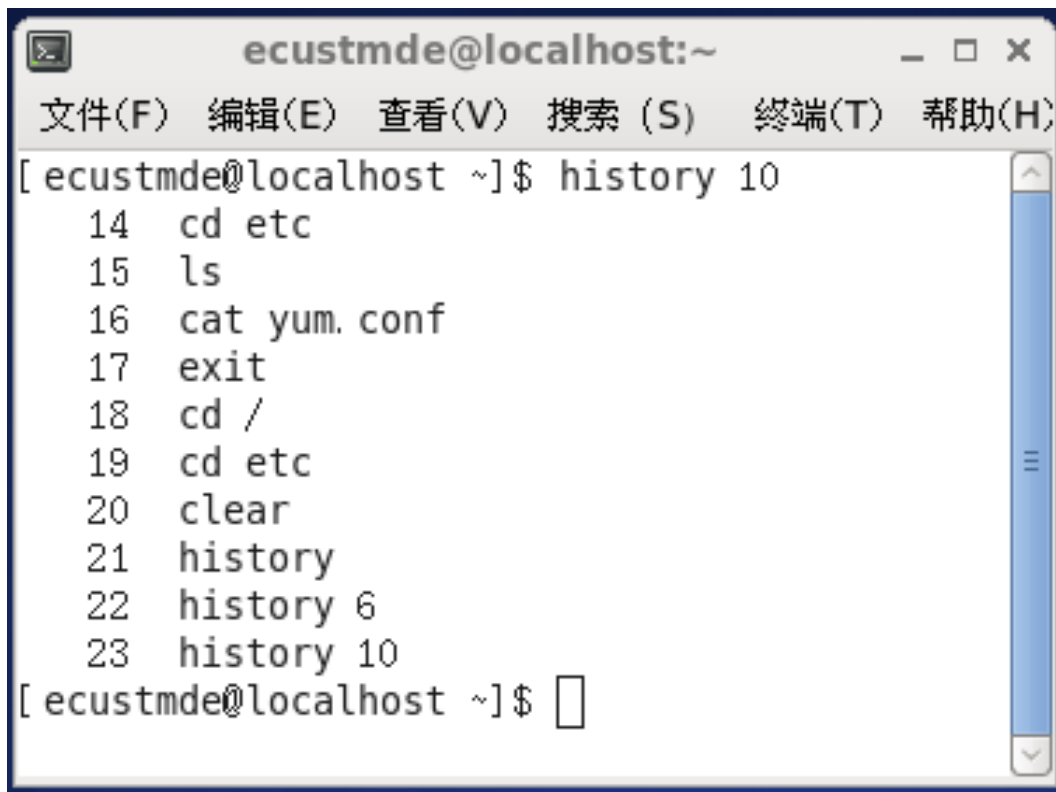
- 可以直接浏览.bash_history文件，或使用history命令来查看目前的命令记录。



```
ecustmde@localhost:~$ cat .bash_history
echo $Shell
echo $SHELL
tcsh
id
who -uH
pwd
echo $HOME
cd
su
exti
exit
cd /
ls
cd etc
ls
cat yum.conf
exit
cd /
cd etc
clear
history
history 6
history 10

ecustmde@localhost:/etc$ history
1 echo $Shell
2 echo $SHELL
3 tcsh
4 id
5 who -uH
6 pwd
7 echo $HOME
8 cd
9 su
10 exti
11 exit
12 cd /
13 ls
14 cd etc
15 ls
16 cat yum.conf
17 exit
18 cd /
19 cd etc
20 clear
21 history
[ecustmde@localhost etc]$
```

- 使用“history n”命令，其中，n表示需要查看的最近执行的命令的条数。
- 例如：列出系统最近执行的10条命令。



```
ecustmde@localhost:~  
文件(F) 编辑(E) 查看(V) 搜索 (S) 终端(T) 帮助(H)  
[ecustmde@localhost ~]$ history 10  
 14 cd etc  
 15 ls  
 16 cat yum.conf  
 17 exit  
 18 cd /  
 19 cd etc  
 20 clear  
 21 history  
 22 history 6  
 23 history 10  
[ecustmde@localhost ~]$
```

运行历史命令：

! n: 运行第n条历史命令

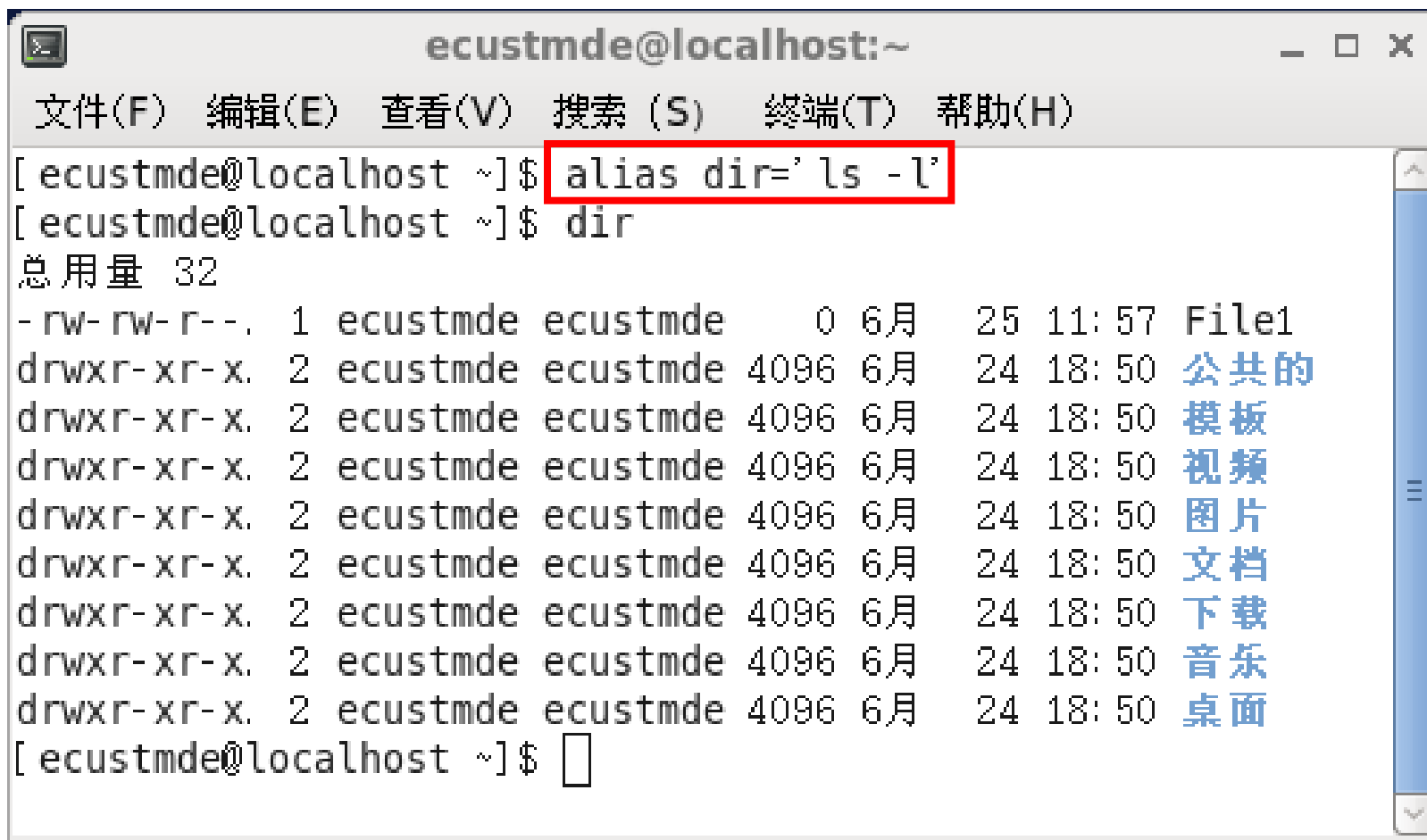
!! : 运行前一条指令

! ?String?: 运行包含string的命令

(4) 别名功能

- 别名功能是指提供给用户使用自定义的简单字符串，来替换复杂的命令选项，或是替换连续多个命令的连续组合的功能，从而使得用户可以自定义符合自己习惯的操作命令。
- 格式： `alias 别名='命令'`

- 创建别名：
 - 例如：如果希望使用dir来代替ls -l，则可以使用alias功能来创建一个到ls -l的别名



A terminal window titled "ecustmde@localhost:~" with a menu bar containing "文件(F)", "编辑(E)", "查看(V)", "搜索 (S)", "终端(T)", and "帮助(H)". The terminal shows the command `alias dir='ls -l'` being entered and highlighted with a red box. Below it, the command `dir` is entered, resulting in a long listing of files and directories. The listing includes permissions, file size, owner, group, modification date, and filename. The filenames are: "File1", "公共的", "模板", "视频", "图片", "文档", "下载", "音乐", and "桌面". The terminal ends with a prompt `[ecustmde@localhost ~]$` and a cursor.

```
ecustmde@localhost:~  
文件(F) 编辑(E) 查看(V) 搜索 (S) 终端(T) 帮助(H)  
[ecustmde@localhost ~]$ alias dir='ls -l'  
[ecustmde@localhost ~]$ dir  
总用量 32  
-rw-rw-r--. 1 ecustmde ecustmde  0 6月 25 11:57 File1  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 公共的  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 模板  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 视频  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 图片  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 文档  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 下载  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 音乐  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 桌面  
[ecustmde@localhost ~]$
```

- 查看别名：如果希望查看当前Linux系统中使用的别名命令，可以直接输入“alias”命令。
- 取消别名：如果需要取消特定的别名命令，可以使用unalias命令。例如，取消dir别名命令可使用如下命令。

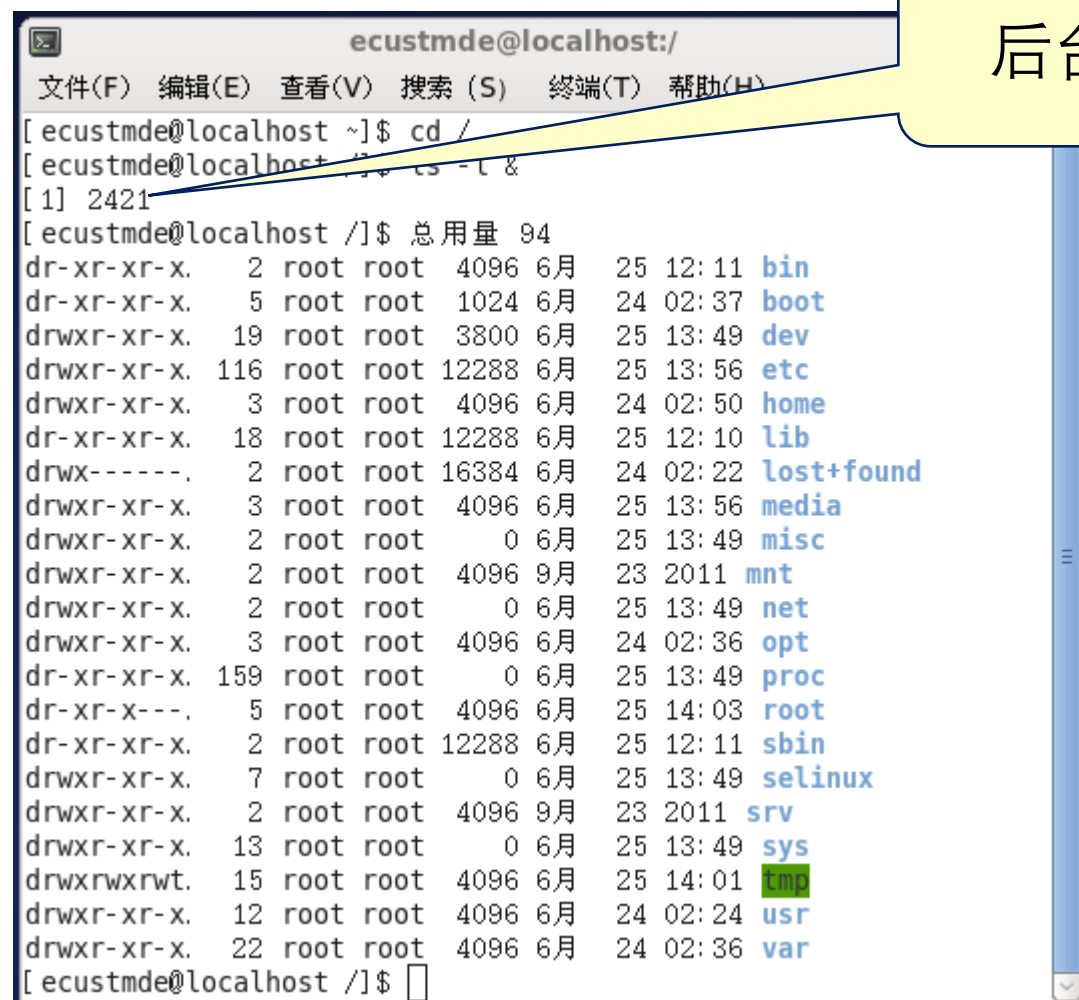
```
# unalias dir [ecustmde@localhost ~]$ alias
alias dir='ls -l'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias vi='vim'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot --show-ti
lde'
[ecustmde@localhost ~]$ unalias dir
[ecustmde@localhost ~]$ alias
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias vi='vim'
alias which='alias | /usr/bin/which --tty-only --read-alias --show-dot --show-ti
lde'
[ecustmde@localhost ~]$ █
```

(5) 后台处理

- Linux是多用户多任务的操作系统，它允许多个用户同时登录系统，也允许多个程序同时执行。但因为shell使用交互式模式，目前执行的命令会一直掌握系统的控制权，直到该程序结束为止，这类程序称为前台程序。

- 只要在输入命令时，在命令后面加上“&”符号。系统就会开始以后台的方式执行该命令，屏幕将显示该后台运行程序的进程PID，然后shell将回到命令提示符状态，以等待用户的下一个命令的输入。
- 格式： 命令 &

- 例如：将 ls-l 命令后台处理。



```
ecustmde@localhost:/  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[ecustmde@localhost ~]$ cd /  
[ecustmde@localhost ~]$ ls -l &  
[1] 2421  
[ecustmde@localhost /]$ 总用量 94  
dr-xr-xr-x.  2 root root  4096 6月  25 12:11 bin  
dr-xr-xr-x.  5 root root  1024 6月  24 02:37 boot  
drwxr-xr-x. 19 root root  3800 6月  25 13:49 dev  
drwxr-xr-x. 116 root root 12288 6月  25 13:56 etc  
drwxr-xr-x.  3 root root  4096 6月  24 02:50 home  
dr-xr-xr-x.  18 root root 12288 6月  25 12:10 lib  
drwx-----  2 root root 16384 6月  24 02:22 lost+found  
drwxr-xr-x.  3 root root  4096 6月  25 13:56 media  
drwxr-xr-x.  2 root root    0 6月  25 13:49 misc  
drwxr-xr-x.  2 root root  4096 9月  23 2011 mnt  
drwxr-xr-x.  2 root root    0 6月  25 13:49 net  
drwxr-xr-x.  3 root root  4096 6月  24 02:36 opt  
dr-xr-xr-x. 159 root root    0 6月  25 13:49 proc  
dr-xr-x---.  5 root root  4096 6月  25 14:03 root  
dr-xr-xr-x.  2 root root 12288 6月  25 12:11/sbin  
drwxr-xr-x.  7 root root    0 6月  25 13:49 selinux  
drwxr-xr-x.  2 root root  4096 9月  23 2011 srv  
drwxr-xr-x. 13 root root    0 6月  25 13:49 sys  
drwxrwxrwt. 15 root root  4096 6月  25 14:01 tmp  
drwxr-xr-x. 12 root root  4096 6月  24 02:24 usr  
drwxr-xr-x. 22 root root  4096 6月  24 02:36 var  
[ecustmde@localhost /]$
```

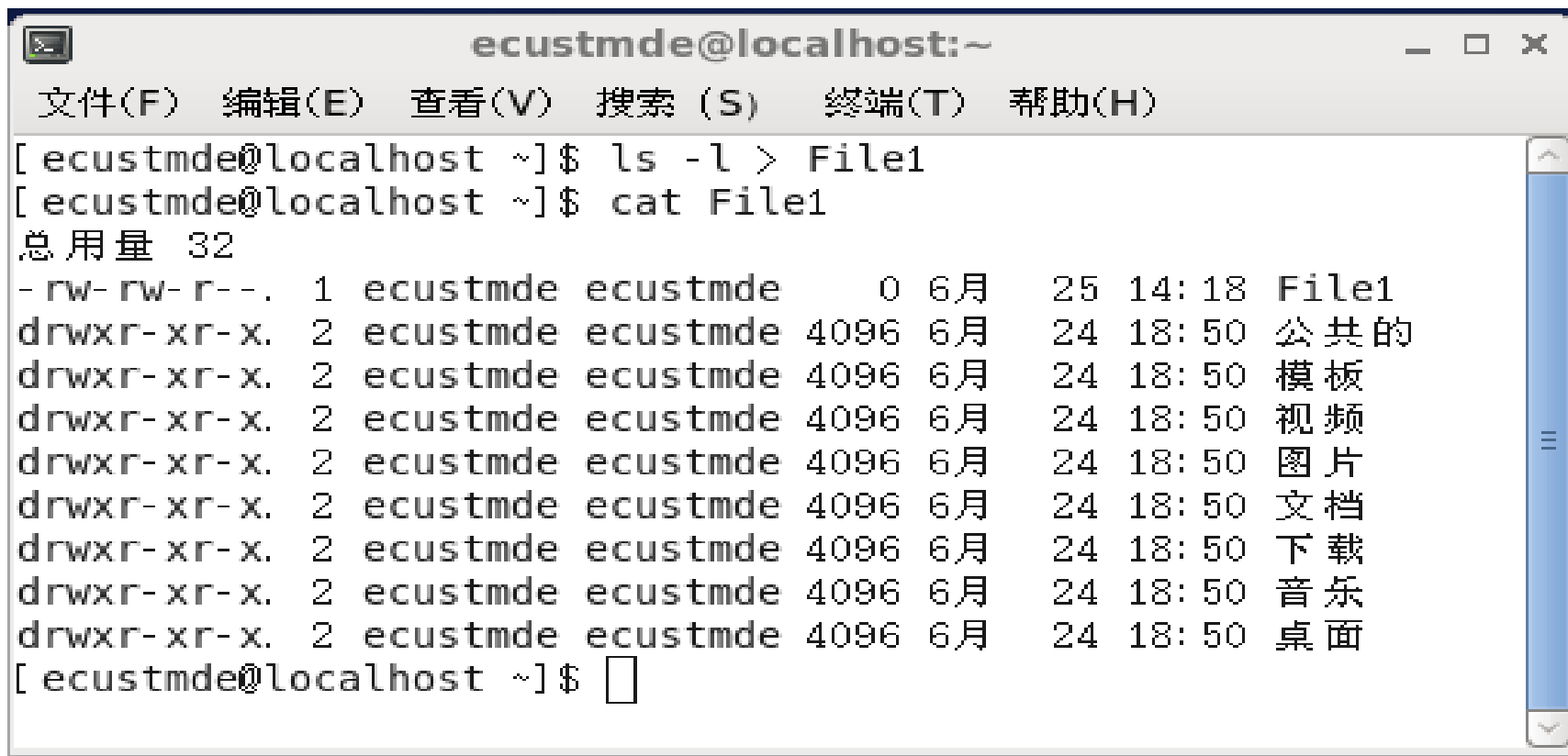
后台进程PID

(6) I/O重定向

标准输入和输出有3种形态：

- 标准输入 (stdin)：通常是指键盘。
- 标准输出 (stdout)：通常是指将命令执行的结果输出到终端机或屏幕上。
- 标准错误输出 (stderr)：是指在命令发生错误时，将其错误信息输出到屏幕上。

- 输出重定向
 - 语法： 命令 > 文件
 - 例如： 将 `ls -l`命令的结果送至File1文件中



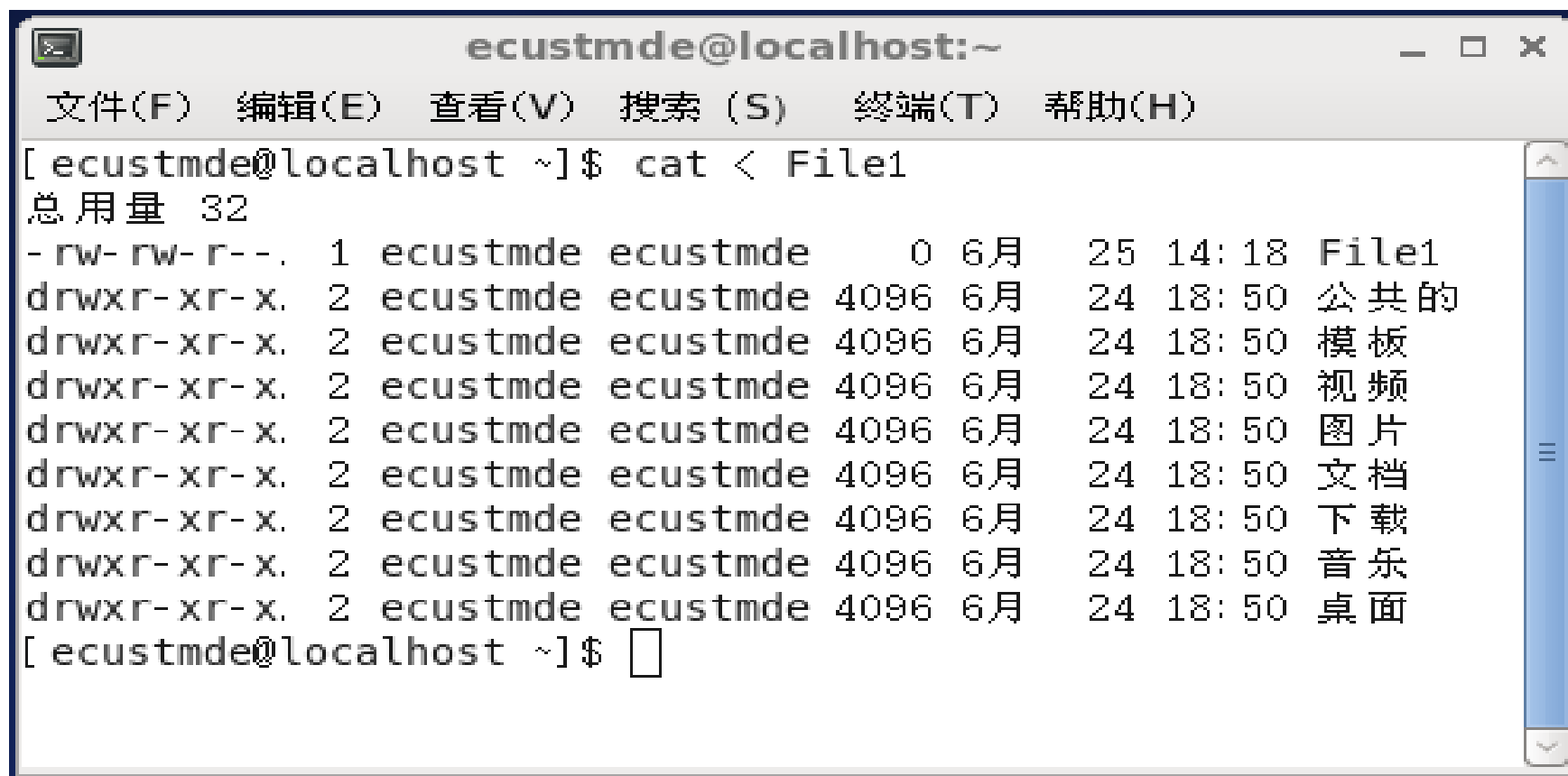
A terminal window titled "ecustmde@localhost:~" with a menu bar containing "文件(F)", "编辑(E)", "查看(V)", "搜索(S)", "终端(T)", and "帮助(H)". The terminal shows the following commands and output:

```
[ecustmde@localhost ~]$ ls -l > File1
[ecustmde@localhost ~]$ cat File1
总用量 32
-rw-rw-r--. 1 ecustmde ecustmde 0 6月 25 14:18 File1
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 公共的
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 模板
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 视频
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 图片
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 文档
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 下载
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 音乐
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 桌面
[ecustmde@localhost ~]$
```

- 输入重定向

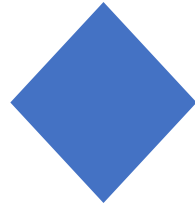
- 语法： 命令 <文件
- 例如： cat从File1获得输入,察看File1文件的内容

从文件，或者其它打开文件以及设备进行输入



A terminal window titled 'ecustmde@localhost:~' with a menu bar containing '文件(F)', '编辑(E)', '查看(V)', '搜索(S)', '终端(T)', and '帮助(H)'. The terminal shows the command '[ecustmde@localhost ~]\$ cat < File1' being executed. The output is a file listing for 'File1' with permissions '-rw-rw-r--', owner 'ecustmde', group 'ecustmde', size '0', and date '6月 25 14:18'. Below this is a list of directory entries with permissions 'drwxr-xr-x', owner 'ecustmde', group 'ecustmde', size '4096', and date '6月 24 18:50'. The entries are '公共的', '模板', '视频', '图片', '文档', '下载', '音乐', and '桌面'. The prompt '[ecustmde@localhost ~]\$' is followed by a cursor.

```
ecustmde@localhost:~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[ ecustmde@localhost ~]$ cat < File1  
总用量 32  
-rw-rw-r--. 1 ecustmde ecustmde    0 6月 25 14:18 File1  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 公共的  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 模板  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 视频  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 图片  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 文档  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 下载  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 音乐  
drwxr-xr-x. 2 ecustmde ecustmde 4096 6月 24 18:50 桌面  
[ ecustmde@localhost ~]$
```



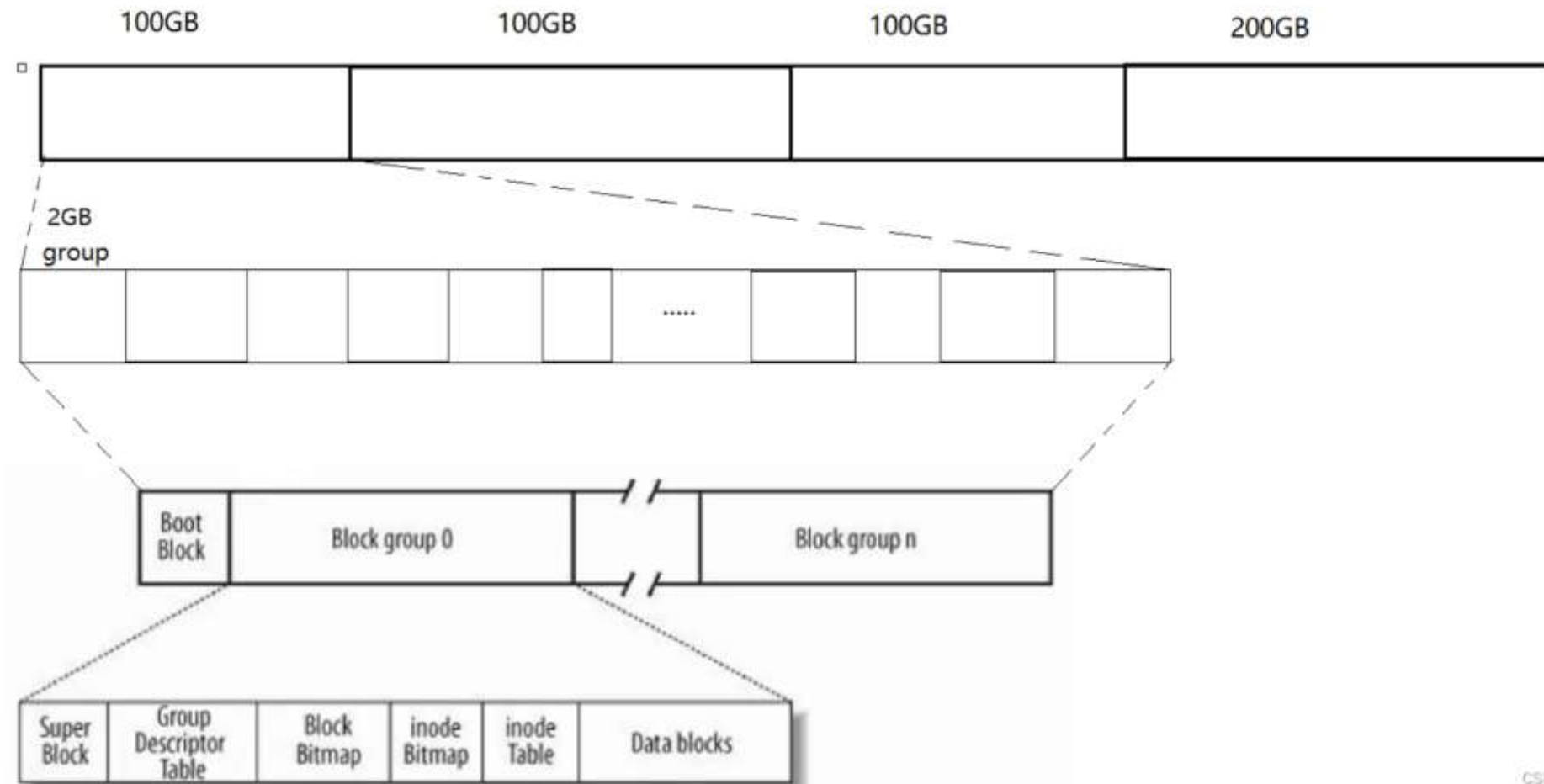
实验二

Linux二级文件系统

- (1) 本实验的目的是通过一个简单多用户文件系统的设计，加深理解文件系统的内部功能和内部实现。
- (2) 结合数据结构、程序设计、计算机原理等课程的知识，设计一个多级目录文件系统，进一步理解操作系统。
- (3) 通过分对实际问题的分析、设计、编程实现，提高学生实际应用、编程的能力。

- 要求（模仿 Linux 的 EXT2 文件系统）
 - 理解和掌握EXT2 文件系统的原理和实现；
 - 创建模拟文件系统（使用二进制文件模拟文件系统），模拟磁盘空间，所有用户信息、inode 信息、超级块信息、文件信息；
 - 实现文件和目录操作：dir、create、delete、open、close、read、write等命令，其中dir列目录时要列出文件名，物理地址，保护码和文件长度
 - 支持用户交互，包括：login用户登录
 - 安全性：源文件可以进行读写保护（读写权限）

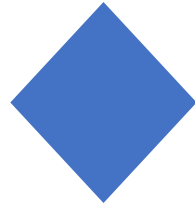
Linux文件系统的设计思路



- Superblock(超级块): 存放文件系统本身的结构信息, 例如: block 和 inode 的总量, 未使用的block和inode的数量, block和inode的大小.....。
- Group Description Table(组描述符): 用来描述每个 组的开始与结束位置的块号码, 以及说明每个块分别介于哪一个 block 号码之间。
- Block bitmap(块位图): 记录哪个数据块已经被占用, 哪个数据块没有被占用。
- Inode bitmap(inode 位图): 记录的是使用与未使用的 inode 号。
- Inode table(inode 表): 存放着所有文件的 inode信息。
- Data Blocks(数据块): 来存放文件内容

Inode节点中存储信息:

- 文件的读写权限(rwx)
- 文件的拥有者和所属组(owner/group)
- 文件的容量该文件的 ctime(创建时间)
- 文件的 atime(最近一次的读取时间)
- 文件的 mtime(最近修改的时间)
- 文件真正内容的指向 (12个直接 + 1个间接 + 个双间接 + 1个三间接记录区)

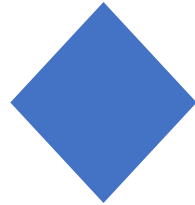


实验三

Linux系统调用

- 学习系统调用的执行流程，理解Linux系统调用
- 设计并实现**进程全量监控与分析系统**，要求设计并实现 3个自定义系统调用和1个用户态监控程序
 - sys_proc_collect: 一次性收集所有进程的完整信息
 - sys_proc_snapshot: 基于进程树，返回父子关系拓扑图
 - sys_proc_stat: 返回系统整体进程统计

- 用户态监控程序：必须通过syscall()调用上述 3 个自定义系统调用，禁止调用ps、/proc文件读取、eBPF、ptrace 等旁路采集手段。
- 具体功能包括：
 - 多条件过滤显示（按PID、进程名、状态）
 - 实时刷新（每秒刷新一次）进程状态快照
 - 进程排序功能（按CPU时间/内存使用量排序）
 - 进程树导出功能（显示带缩进的父子进程树，导出至本地文本文件）

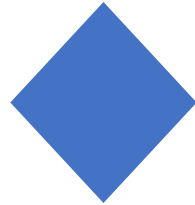


实验四

Linux 驱动程序

- 理解Linux内核模块化机制、设备驱动注册原理、内核态与用户态数据交互方式，掌握无重启内核的内核功能拓展技术
- 开发一个**虚拟字符设备驱动程序**，实现设备的open、read、write、close内核回调函数。
- 备注：实验也可以实现其他任意设备的驱动程序

- 包括设备号管理、字符设备注册、自动创建设备文件等模块，要求：
 - 用户态可向虚拟设备写入数据，内核态接收数据并处理，可读取内核返回的设备状态信息。
 - 驱动内置一页大小内核缓冲区，用户 read 读缓冲区、write 写入缓冲区；
 - 支持多进程打开同一设备，独立读写偏移；
 - 支持自动分配设备号、自动创建设备文件；
 - 支持模块动态加载、卸载、设备节点注册测试。



实验五

其他

- 操作系统中的软件设计
 - 包括针对现有的各种类型操作系统(Linux, Android)中的设备管理、存储管理、用户接口、文件系统管理功能进行改进或者剪裁。